แบบฝึกหัดปฏิบัติการคาบที่ 8: Interface / Abstract

2. ให้ศึกษาและทดลองพิมพ์ Code ตัวอย่างการสร้างอินเตอร์เฟสจากตัวอย่างต่อไปนี้

```
public class House implements Cloneable, Comparable<House> {
    private int id;
    private double area;
    private java.util.Date whenBuilt;

    public House(int id, double area) {
        this.id = id;
        this.area = area;
        whenBuilt = new java.util.Date();
    }

    public int getId() {
        return id;
    }

    public double getArea() {
        return area;
    }

    public java.util.Date getWhenBuilt() {
        return whenBuilt;
    }

    @Override
    public Object clone() {
        try {
            return super.clone();
        } catch (CloneNotSupportedException ex) {
            return null;
        }
    }

    @Override
    public int compareTo(House o) {
        if (area > o.area)
            return 1;
        else if (area < o.area)
            return -1;
        else
            return 0;
    }
}
```

2.1 ให้อธิบายการทำงานของโปรแกรมต่อไปนี้

บรรทัดที่ 1

ประกาศคลาส House ซึ่ง implements Cloneable เพื่อให้สามารถคัดลอก object ได้ และ implements Comparable<House> เพื่อให้สามารถเปรียบเทียบ object House ตามพื้นที่ได้

บรรทัดที่ 2- 4

ประกาศตัวแปร id สำหรับเก็บรหัสบ้าน, area สำหรับเก็บพื้นที่บ้าน และ whenBuilt สำหรับเก็บวันที่และเวลาที่สร้าง object บ้าน โดยกำหนดให้ตัวแปรทั้งหมดเป็น private

บรรทัดที่ 5- 9

เป็น constructor ของคลาส House รับค่า id และ area มากำหนดให้กับตัวแปรของ object และสร้าง object Date เพื่อเก็บวันที่และเวลาปัจจุบันไว้ใน whenBuilt

บรรทัดที่ 10-18

เป็น getter methods สำหรับคืนค่าของ id, area และ whenBuilt ทำให้คลาสอื่นสามารถเข้าถึงค่าของตัวแปรเหล่านี้ได้ผ่านเมธอดที่กำหนด

บรรทัดที่ 21-28

เป็นเมธอด clone() สำหรับคัดลอก object House โดยเรียก super.clone() เพื่อสร้างสำเนาแบบ shallow copy หากเกิด CloneNotSupportedException จะคืนค่า null

บรรทัดที่ 30-37

เป็นเมธอด compareTo(House o) สำหรับเปรียบเทียบพื้นที่ (area) ของบ้านสองหลัง โดยคืนค่า 1 หากบ้านปัจจุบันมีพื้นที่มากกว่า -1 หากมีพื้นที่น้อยกว่า และ 0 หากมีพื้นที่เท่ากัน

2.2 สร้างคลาสทดสอบ และเพิ่มคำสั่งต่อไปนี้ใน main() ผลลัพธ์จะมีผลลัพธ์อย่างไร

```
House house1 = new House(1, 1750.50);
House house2 = (House)house1.clone();
```

คลาสทดสอบเต็ม:

```
public class TestHouse {
    public static void main(String[] args) {
        House house1 = new House(1, 1750.50);
        House house2 = (House) house1.clone();

        System.out.println(house1 == house2);
        System.out.println(house1.getId());
        System.out.println(house2.getId());
        System.out.println(house1.getArea());
        System.out.println(house2.getArea());
        System.out.println(house1.getWhenBuilt() == house2.getWhenBuilt());
    }
}
```

Output:

```
false
1
1
1750.5
1750.5
true
```

3. จงเขียน class ที่วาดกราฟจากฟังก์ชันต่อไปนี้

```
abstract class AbstractDrawFunction extends JPanel
{
  /**Polygon to hold the points*/
  private Polygon p = new Polygon();
  /**Default constructor*/
  protected AbstractDrawFunction ()
  {
    drawFunction();
    setBackground(Color.white);
  }
  /**Draw the function*/
  public abstract double f(double x);
  /**Obtain points for x coordinates 100, 101, ..., 300*/
  public void drawFunction()
  {
    for (int x = -100; x <= 100; x++)
    {
      p.addPoint(x+200, 200-(int)f(x));
    }
  }
  /**Paint the function diagram*/
  public void paintComponent(Graphics g)
  {
    super.paintComponent(g);
    // Draw x axis
    g.drawLine(10, 200, 390, 200);
   // Draw y axis
    g.drawLine(200,30, 200, 390);
    // Draw arrows on x axis
    g.drawLine(390, 200, 370, 190);
    g.drawLine(390, 200, 370, 210);
   // Draw arrows on y axis
    g.drawLine(200, 30, 190, 50);
    g.drawLine(200, 30, 210, 50);
    // Draw x, y
    g.drawString("X", 370, 170);
    g.drawString("Y", 220, 40);
    // Draw a polygon line by connecting the points in the polygon
    g.drawPolyline(p.xpoints, p.ypoints, p.npoints);
  }
}
```

ทดสอบคลาสโดยใช้ฟังก์ชันต่อไปนี้

a. $f(x) = x^2;$

b. $f(x) = \sin(x);$

c. $f(x) = \cos(x);$

d. $f(x) = \tan(x);$

e. $f(x) = \cos(x) + 5\sin(x);$

f. $f(x) = 5\cos(x) + \sin(x);$

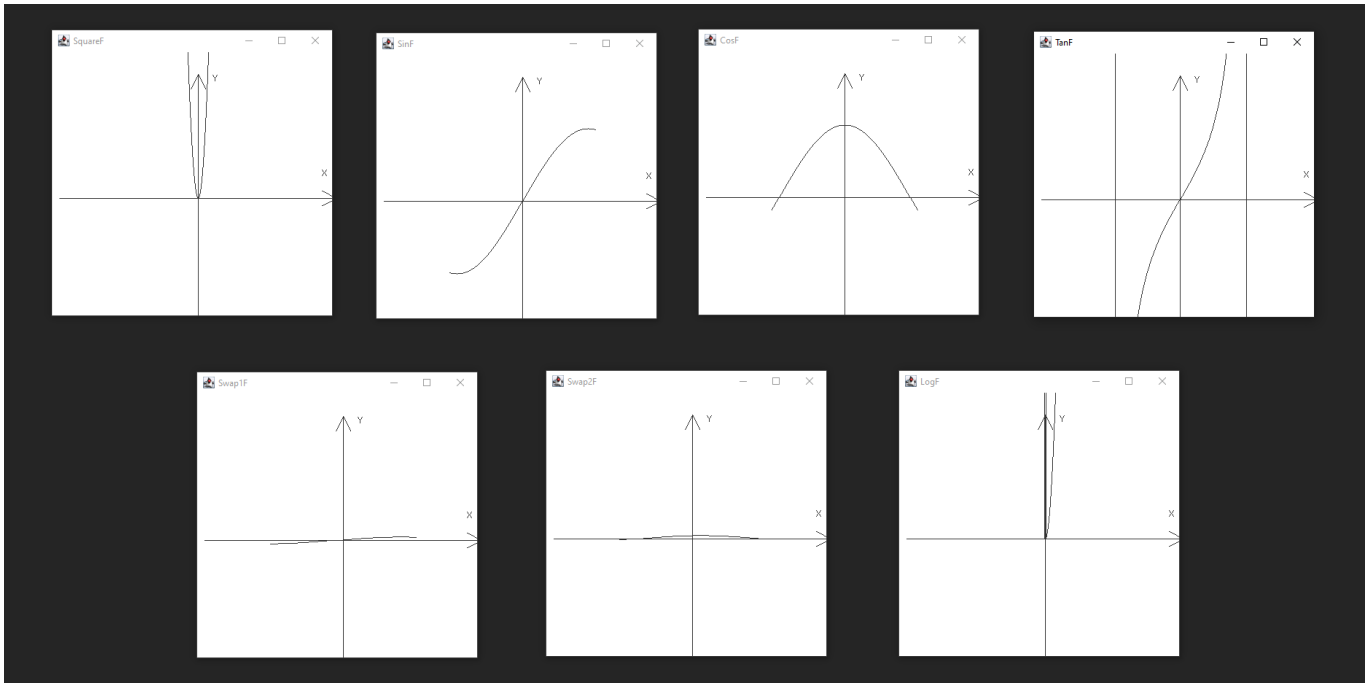
g. $f(x) = \log(x) + x^2;$

สำหรับฟังก์ชันแต่ละฟังก์ชันให้สร้างคลาสที่ extends คลาส AbstractDrawFunction และ implements เมธอด f(double x);

กำหนดให้คลาสทดสอบคือ

```
public class Test extends JFrame
{
  public Test()
  {
    getContentPane().setLayout(new GridLayout(1, 2, 5, 5));
    getContentPane().add(new DrawSine());
  }
  public static void main(String[] args)
  {
    Test frame = new Test();
    frame.setSize(400, 400);
    frame.setTitle("Exercise 10.10");
    frame.setVisible(true);
  }
}
```

ตัวอย่างภาพแสดง:



5. กำหนดให้ GeometricObject.java มีรายละเอียดดังนี้

```
public abstract class GeometricObject {
  private String color = "white";
  private boolean filled;
  protected GeometricObject() {
  }
  protected GeometricObject(String color, boolean filled) {
    this.color = color;
    this.filled = filled;
  }
  public String getColor() {
    return color;
  }
  public void setColor(String color) {
    this.color = color;
  }
  public boolean isFilled() {
    return filled;
  }
  public void setFilled(boolean filled) {
    this.filled = filled;
  }
  public abstract double getArea();
  public abstract double getPerimeter();
}
```

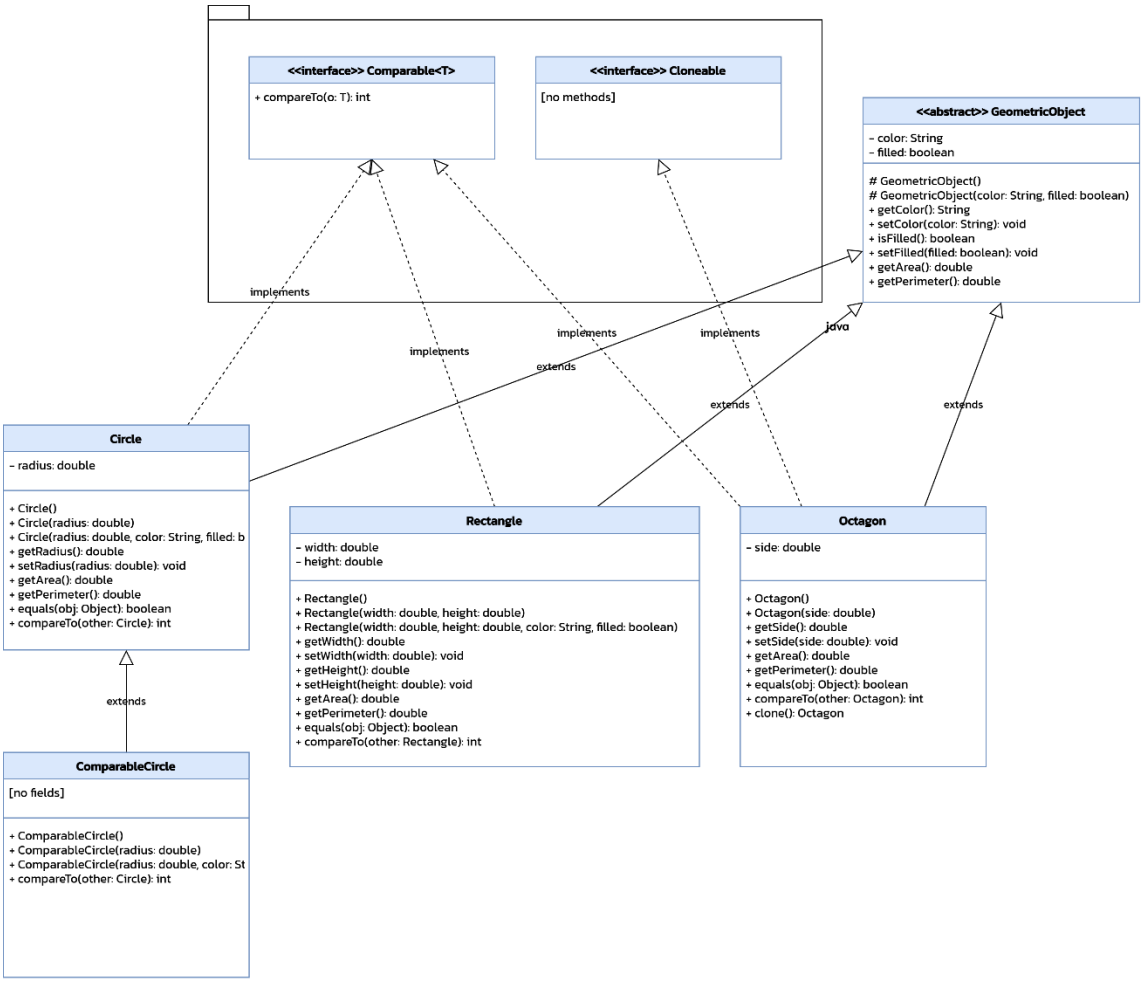
5.1 ให้สร้างคลาส Circle ที่ extend GeometricObject และ implements อินเตอร์เฟส Comparable ให้ทำการ override เมธอด equals() ของคลาส Object และ implements เมธอด compareTo() เพื่อทำการเปรียบเทียบวงกลมจากรัศมีของวงกลม โดยที่วงกลมสองวงจะเท่ากันก็ต่อเมื่อมีรัศมีเท่ากัน วาดคลาสไดอะแกรมและเขียนโปรแกรมทดสอบคลาสดังกล่าว

5.2 ให้สร้างคลาส Rectangle ที่ extends GeometricObject และ implements อินเตอร์เฟส Comparable ให้ทำการ override เมธอด equals() ของคลาส Object และ implements เมธอด compareTo() เพื่อทำการเปรียบเทียบ Rectangle จากพื้นที่ของ Rectangle โดยที่สี่เหลี่ยมจะเท่ากันก็ต่อเมื่อมีพื้นที่เท่ากัน วาดคลาสไดอะแกรมและเขียนโปรแกรมทดสอบคลาสดังกล่าว

5.3 ให้สร้างคลาส Octagon ที่ extends GeometricObject และ implements อินเตอร์เฟส Comparable และ Cloneble ให้ทำการ override เมธอด equals() ของคลาส Object เมธอด compareTo() เพื่อทำการเปรียบเทียบ Octagon จากพื้นที่ของ Octagon โดยที่รูปแปดเหลี่ยมจะเท่ากันก็ต่อเมื่อมีพื้นที่เท่ากัน โดย พื้นที่ของ Octagon คือ $\boxed{\text{area} = \text{side}^2\left(2+\frac{4}{\sqrt{2}}\right)}$ วาดคลาสไดอะแกรมและเขียนโปรแกรมทดสอบคลาสดังกล่าวโดยสร้าง Object จากคลาส Octagon ที่มี side=5 ให้แสดงพื้นที่และเส้นรอบวง หลังจากนั้นให้สร้าง object ใหม่โดยการใช้เมธอด clone และเปรียบเทียบว่า object ทั้งสองดังกล่าวเท่ากันหรือไม่

5.4 สร้างคลาสที่ชื่อว่า ComparableCircle ที่สืบทอดมาจากคลาส Circle และ implements อินเตอร์เฟส Comparable วาด UML class diagram และ implements เมธอด compareTo() เพื่อทำการเปรียบเทียบวงกลมจากพื้นที่ของวงกลม เขียน Test class ที่ค้นหาวงกลมที่ใหญ่ที่สุดจาก 2 วงกลมที่เป็น object ของ ComparableCircle

Class Diagram:



Output ของ P05_Main.java:

```
=== 5.1 Circle ===
Circle[radius=5.0, area=78.53981633974483]
c1 equals c2: true
c1 equals c3: false
Compare c1 and c3: -1

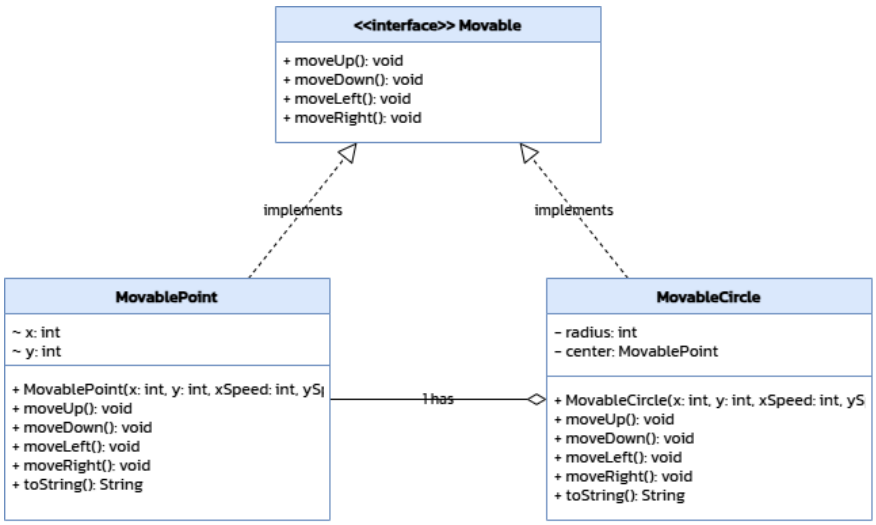
=== 5.2 Rectangle ===
Rectangle[width=4.0, height=6.0, area=24.0]
r1 equals r2: true
r1 equals r3: false
Compare r1 and r3: -1

=== 5.3 Octagon ===
Area: 120.71067811865476
Perimeter: 40.0
Original: Octagon[side=5.0, area=120.71067811865476, perimeter=40.0]
Copy: Octagon[side=5.0, area=120.71067811865476, perimeter=40.0]
Equal: true
Same object: false
```

```
=== 5.4 ComparableCircle ===
Circle 1 area: 78.53981633974483
Circle 2 area: 201.06192982974676
Circle 2 is larger.
```

6. Suppose that we have a set of objects with some common behaviors: they could move up, down, left or right. The exact behaviors (such as how to move and how far to move) depend on the objects themselves. One common way to model these common behaviors is to define an *interface* called Movable, with abstract methods moveUp(), moveDown(), moveLeft() and moveRight(). The classes that implement the Movable interface will provide actual implementation to these abstract methods.

Let's write two concrete classes - MovablePoint and MovableCircle - that implement the Movable interface.



Output ของ P06_Movable.java:

```
Point: (1,2), speed=(3,4)
Move up: (1,-2), speed=(3,4)
Move right: (4,-2), speed=(3,4)

Circle: MovableCircle[center=(5,10), speed=(2,3), radius=7]
Move up: MovableCircle[center=(5,7), speed=(2,3), radius=7]
Move left: MovableCircle[center=(3,7), speed=(2,3), radius=7]
```